

ReinforceGen: Hybrid Skill Policies with Automated Data Generation and Reinforcement Learning

Anonymous Authors

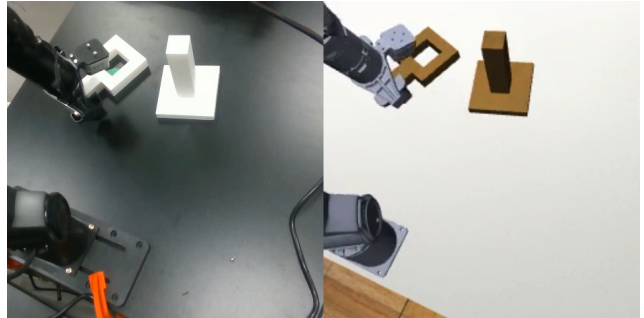
Abstract—Long-horizon manipulation has been a long-standing challenge in the robotics community. We propose ReinforceGen, a system that combines task decomposition, data generation, imitation learning, and motion planning to form an initial solution, and improves each component through reinforcement-learning-based fine-tuning. ReinforceGen first segments the task into multiple localized skills, which are connected through motion planning. The skills and motion planning targets are trained with imitation learning on a dataset generated from 10 human demonstrations, and then fine-tuned through online adaptation and reinforcement learning. When benchmarked on the Robosuite dataset, ReinforceGen reaches 80% success rate on all tasks with visuomotor controls in the highest reset range setting. Additional ablation studies show that our fine-tuning approaches contribute to an 89% average performance increase. Finally, ReinforceGen demonstrates significant improvement through fine-tuning in our real-world evaluations. More results and videos are available at <https://reinforcegenicra27.github.io/>.

I. INTRODUCTION

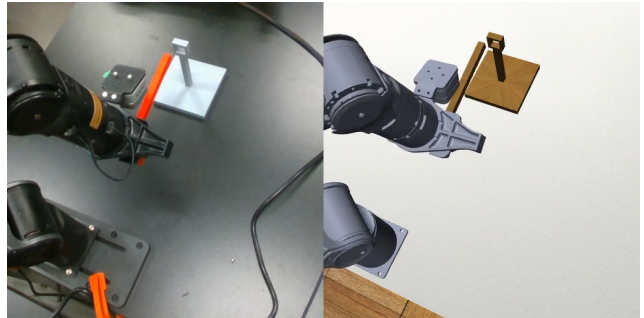
Imitation Learning (IL) from demonstrations is an effective approach for robots to autonomously complete tasks. In long-horizon tasks, collecting demonstrations can be expensive, and the trained agent is more likely to deviate from the demonstrations to out-of-distribution states. Reinforcement Learning (RL) leverages random exploration, incorporating environmental feedback through rewards. However, long horizons exacerbate the exploration challenge, especially when the reward signals are also sparse. In the context of robot learning, collecting demonstrations for IL is often time-consuming and expensive, as it typically requires a teleoperation platform and coordinating with human operators. Furthermore, the solution quality and data coverage of the demonstrations are critical, as they directly impact the agent’s performance when used for Behavior Cloning (BC) [1].

One approach to combat demonstration insufficiency is to augment the dataset through synthetic data generation. In robotic manipulation tasks, a line of work [2–4] focuses on object-centric data generation through demonstration adaptation. Other approaches [5, 6] use Task and Motion Planning (TAMP) [7] to generate demonstrations. An alternative strategy is to hierarchically divide the task into consecutive stages with easier-to-solve subgoals [3, 8, 9]. In most manipulation tasks, only a small fraction of robot execution requires high-precision movements, for example, only the contact-rich segments. Thus, these approaches concentrate the demo collection effort at the precision-intensive skill segments and connect segments using planning, ultimately improving demo sample efficiency.

Still, these demonstration generation methods are open-loop



(a) The *Square* task: the robot inserts a square nut onto a peg.



(b) The *Threading* task: the robot threads a needle through a ring.

Fig. 1: ReinforceGen agents solve long-horizon manipulation tasks through a hybrid planning and learning policy, pretrained with imitation learning of synthetically generated data and fine-tuned with reinforcement learning. *Left:* ReinforceGen solving two contact-rich real-world tasks. *Right:* simulated versions of these tasks.

and rely solely on offline data. As a result, IL agents trained with the generated data are still bottlenecked by the quality of the source demonstrations. To combat this, we propose ReinforceGen, a framework that improves hierarchical data generation by incorporating online exploration and environmental feedback using RL. ReinforceGen trains a hybrid BC agent with object-centric data generation as its base policy. It then combines distillation, causal inference, and RL to improve the base agent with online data, as well as real-time adaptation from environment feedback during deployment. We demonstrate that ReinforceGen produces high-performance hybrid data generators and show that ReinforceGen-generated data can also be used to train end-to-end visuomotor imitation policies. We demonstrate that ReinforceGen can be effectively deployed on a real robot through sim-to-real transfer.

The contributions of this paper are the following.

- We propose ReinforceGen, an automated demonstration generation system that integrates planning, behavior cloning, and reinforcement learning to train policies that robustly solve long-horizon and contact-rich tasks

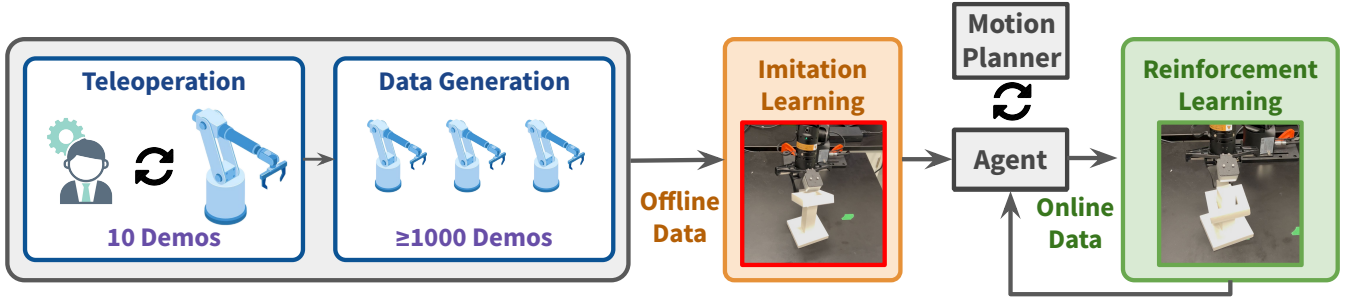


Fig. 2: ReinforceGen first creates an offline dataset by synthetic data generation from a small set of source human demonstrations. The dataset is then used to train a hybrid imitation learning agent that alternates between moving to a predicted waypoint using a motion planner and directly controlling the robot using a learned policy. Finally, ReinforceGen uses reinforcement learning to fine-tune the agent with online environment interactions.

using only a handful of human demonstrations.

- We propose novel integral mechanics that refine the base policy with online interactions, including trajectory re-planning (§IV-A), localized reinforcement learning (§IV-B), and causal termination fine-tuning (§IV-C). Through exploration, ReinforceGen goes beyond existing automated demonstration systems, which are bounded by the performance of the demonstrator.
- We evaluate ReinforceGen on multi-stage contact-rich tasks, where it reaches an **80%** success rate, doubling the success rate of the prior state-of-the-art [3].
- Using sim-to-real transfer, we deploy ReinforceGen on a real robot where it achieves a **70%** success rate, a $2.8\times$ improvement over prior state-of-the-art.

II. RELATED WORK

Imitation and reinforcement learning for robotics. IL from human demonstrations has been a pillar in robot learning, benchmarking significant results in both simulations and real-world applications [10–14]. RL is another promising approach to solving robotic problems with even real-world successes [15–17]. However, without engineered rewards, RL often struggles with exploration and can even outright fail [9]. Similar to our approach, Nair et al. [18], Johannink et al. [19], and Zhou et al. [9] investigate combining the strengths of IL and RL, using the IL policy as a warm start for RL exploration and using RL to fine-tune the IL agent in out-of-distribution states. However, ReinforceGen also leverages motion planning to decompose control into smaller skill-learning subproblems, which decreases the horizon required for RL to explore and thus the state space, boosting policy success rates.

Automated data generation. McDonald et al. [5] and Dalal et al. [6] use Task and Motion Planning (TAMP) [7] to generate demonstrations for imitation learning; however, these systems focus on prehensile manipulation due to the difficulty of modeling contact-rich skills. Dalal et al. [20] leverage Large Language Models (LLMs) for task planning in place of a full planning model. MimicGen [2], SkillMimicGen [3], DexMimicGen [4], DemoGen[21], CP-Gen [22], and MoMa-Gen [23] automatically bootstrap a dataset of demonstrations from a few annotated human source demonstrations through pose adaptation and trial-and-error execution. A key drawback is that the generated dataset has limited diversity because

each demonstration is derived from the same small set. In ReinforceGen, we address this by using reinforcement learning to explore beyond these demonstrations in search of more successful and less costly behaviors.

Hybrid planning and learning systems. Several approaches integrate learned policies into TAMP systems. Wang et al. [24] engineered open-loop parameterized skill policies and learned successful skill parameters using Gaussian Process regression. NOD-TAMP [25] used Neural Object Descriptors (NODs) to warp human skill demonstrations to new object geometries and deploy them open-loop within TAMP. HITL-TAMP [8] learned closed-loop visuomotor policies in place of open-loop skills using BC. The closest to our approach is SPIRE [9], which goes beyond HITL-TAMP by using RL to fine-tune initiation-learned skills, improving their success rates and decreasing execution costs. In order to leverage TAMP, these approaches assume access to an engineered planning model that specifies the preconditions and effects of each skill. In contrast, ReinforceGen learns and fine-tunes initiation and termination models that implicitly encode these dynamics without prior information.

III. PRELIMINARIES

We model manipulation tasks as Partially Observable Markov Decision Processes (POMDPs) where $\mathcal{S}$ is the state space, $\mathcal{O}$ is the observation space, $\mathcal{A}$ is the action space, and $\mathcal{T} \subseteq \mathcal{S}$ is a set of terminal states. We are interested in producing a policy $\pi : \mathcal{O} \rightarrow \mathcal{A}$ that controls the system from initial state $s_0 \in \mathcal{S}$ to a terminal state $s_T \in \mathcal{T}$ while minimizing the number of steps taken.

A. Object-Centric Data Generation

ReinforceGen builds on MimicGen [2], which generates data by transforming and replaying human demonstrations for manipulation tasks. It assumes the environment contains a set of manipulable objects $M = \{O_1, \dots, O_m\}$, and their poses comprise a component of states $s \in \mathcal{S}$. MimicGen divides the task into multiple contiguous object-centric subtask segments, where each segment i is associated with a fixed reference object $R_i \in M$. Let $T_B^A \in \text{SE}(3)$ be the homogeneous transformation matrix representing object A 's pose in the frame of object B , where $A, B \in M$ and W is the world frame. For a demonstration involving object A , the source trajectories of end-effector poses $\{T_W^{A_i}\}$ are parsed into

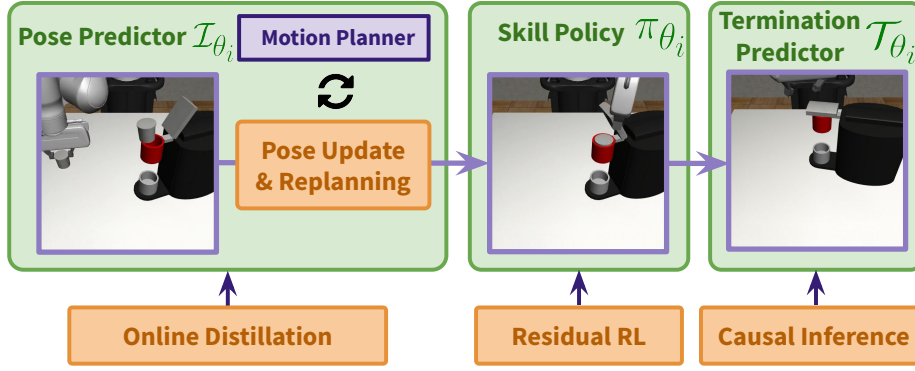


Fig. 3: The three main components of a ReinforceGen stage. The pose predictor $\mathcal{I}_{\theta_i}$ predicts the target end-effector pose and updates the goal of the motion planner in real-time. After reaching the destination, the skill policy π_{θ_i} takes control to complete the stage goal, which is given by the termination predictor $\mathcal{T}_{\theta_i}$. Each component is trained offline with supervised learning and then fine-tuned with online data.

the frame of the corresponding reference objects $T_{R_i}^{A_t} \leftarrow (T_W^{R_i})^{-1} T_W^{A_t}$. To generate new data in a new environment instantiation, segments of a sampled source trajectory are adapted according to the current reference object R_i' pose $T_W^{A_t} \leftarrow T_W^{R_i'} T_{R_i}^{A_t}$. Delta pose actions are extracted from the transformed poses and executed open loop along with the original gripper actions. After execution, trajectories that reach a goal state comprise the new synthetic dataset.

B. Hybrid Skill Policy

ReinforceGen adopts the bilevel decomposition from the Hybrid Skill Policy (HSP) framework [3] to reduce the complexity of long-horizon tasks by framing control as a sequence of connected contact-rich subproblems. More formally, an HSP decomposes the task into n stages, where each stage comprises a connect segment followed by a skill segment. In the connect segment, the robot moves from its current position to the initiation position of the skill segment. Following the *options* framework [26], the i -th skill in the skill sequence $\psi_i := \langle \mathcal{I}_i, \pi_i, \mathcal{T}_i \rangle$ consists of an initiation end-effector pose condition $\mathcal{I}_i \subseteq \text{SE}(3)$, a skill policy $\pi_i : \mathcal{O} \rightarrow \mathcal{A}$, and a termination condition $\mathcal{T}_i \subseteq \mathcal{S}$. When applying this formation to object-centric data generation (Sec. III-A), we assume that the skill segment in each stage uses a reference object $R_i \in M$. Initiation end-effector poses can also be generated in an object-centric fashion by transforming the target pose E_0 in the source trajectory in the frame of the original reference object to the current: $T_W^{E_0'} \leftarrow T_W^{R_i'} T_{R_i}^{E_0}$, as in [3].

States are not directly observable at deployment time, so we seek to learn parametric models of skills that operate only on observations. Formally, an HSP *agent* is a sequence of parameterized skills $[\psi_{\theta_1}, \dots, \psi_{\theta_n}]$. Each skill $\psi_{\theta_i} := \langle \mathcal{I}_{\theta_i}, \pi_{\theta_i}, \mathcal{T}_{\theta_i} \rangle$ comprises an *initiation predictor* $\mathcal{I}_{\theta_i} : \mathcal{O} \rightarrow \text{SE}(3)$ that predicts a skill start pose in $\mathcal{I}_i$ based on the current observation, a parameterized *skill policy* $\pi_{\theta_i} : \mathcal{O} \rightarrow \mathcal{A}$, and a *termination classifier* $\mathcal{T}_{\theta_i} : \mathcal{O} \rightarrow \{0, 1\}$ that predicts whether the current state is a terminal state in $\mathcal{T}_i$. The parameters $\{\theta_i\}$ are learned through behavior cloning, maximizing the likelihood of producing the generated data: $P(E_0' = \mathcal{I}_{\theta_i}(o_0))$, $P(a_t' = \pi_{\theta_i}(o_t))$, and $P([s_t \in \mathcal{T}_i] = \mathcal{T}_{\theta_i}(o_t))$. At each stage,

an HSP predicts a starting pose with $\mathcal{I}_{\theta_i}(o_0)$, moves to the pose with motion planning, and executes the skill policy π_{θ_i} until $\mathcal{T}_{\theta_i}$ predicts termination. As in prior work [2, 3], we assume the stage sequence and reference objects R_i are annotated by a human.

IV. REINFORCEGEN

Existing object-centric data generation methods [2, 3, 8] have been shown to be capable of generating high-quality trajectory datasets that produce performant policies in long-horizon manipulation tasks from a small set of human demonstrations. Despite their impressive performance, these methods are still bounded by the quality of the demonstrations since they primarily replay transformed source trajectories or directly run IL on the collected demonstrations, without exploring new behaviors. ReinforceGen presents a solution to improve data generation quality substantially by integrating a trained HSP imitation learning policy (Sec. III-B) and online environment feedback into the data generation workflow, allowing the policy, and ultimately the generated data, to improve over time via exploration and exploitation. Specifically, we propose fine-tuning pipelines for individual parameterized components of an HSP with online adaptation and reinforcement learning, namely the initiation pose predictor $\mathcal{I}_{\theta_i}$ (Sec. IV-A), skill policy π_{θ_i} (Sec. IV-B), and termination predictor $\mathcal{T}_{\theta_i}$ (Sec. IV-C). With the fine-tuned components, we execute these skill policies in a hybrid policy that alternates between motion planning and the skills (Sec. IV-D). Finally, we demonstrate the ability to optionally distill the hybrid policy to a fully end-to-end policy that does not rely on a motion planner (Sec. IV-E). We illustrate the fine-tuning modules in Fig. 4.

A. Initiation Pose Prediction

The initiation pose predictor $\mathcal{I}_{\theta_i}$ proposes a target pose for the motion planner to reach before handing control off to the skill policy. It plays a critical role in the HSP framework since it directly dictates the initial state distribution of the skill policy. Even small errors can lead to out-of-distribution states and cause severe performance degradation. To illustrate this, we performed an experiment where we artificially add noise to the initiation pose with increasing scales. Fig. 5 plots

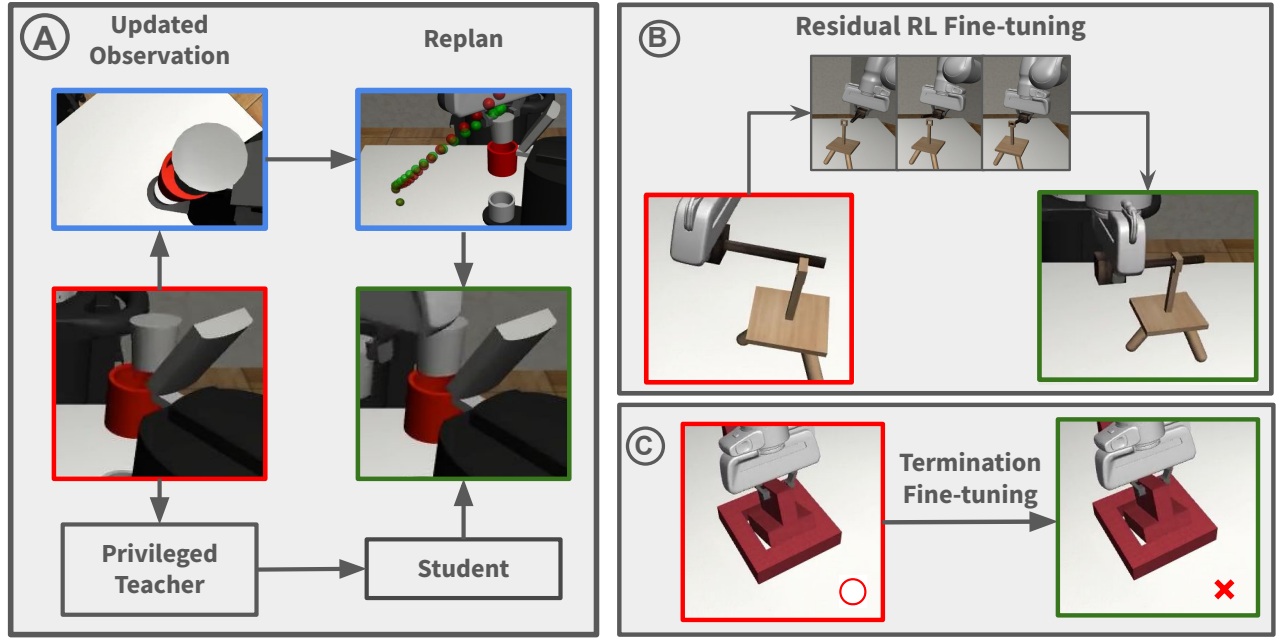


Fig. 4: Depiction of how we fine-tune the three components. (A) The pose predictor based on image and robot state input is fine-tuned towards a teacher predictor with privileged object state information. During execution, it constantly updates its prediction when gathering new information about the target object and reroutes dynamically. (B) The skill policy is fine-tuned through residual reinforcement learning. (C) We purge the false-positive stage termination predictions that lower the completion rate of later stages.

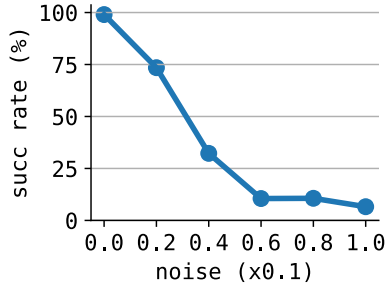


Fig. 5: The policy success rate drops sharply as the pose target noise increases in the second stage of *Nut Assembly*.

the success rate of the following skill policy as the noise increases. As expected, the success rate drops sharply as the prediction error increases. In practice, such errors can often occur, especially when the agent has imperfect perception. To mitigate this, we propose two methods that utilize online interactions during deployment and in training.

Real-time replanning. As the robot arm approaches the next skill, more information can be gathered to refine the initiation pose. Typically, a robot equipped with a wrist camera can only capture the target location when the end-effector is close to it. Therefore, instead of running the pose predictor once, we run it at every timestep of the transit phase, while executing a motion plan. When the difference between the newly predicted pose and the previously planned target $T_W^{E_0}$ exceeds a given threshold, we then restart motion planning with the updated pose.

Student predictor from a privileged teacher. The replanning approach stresses the importance of continuously making predictions during motion planning. However, when

using a suboptimal predictor, the motion planning trajectory will inevitably encounter out-of-distribution states, thereby hindering prediction accuracy. During training, we have access to privileged object state information that allows us to use an accurate teacher predictor. Specifically, the privileged predictor selects one of the source demos and transforms the target pose in the demo according to the current object state, similar to the data generation procedure described in Sec. III-A. We denote the teacher predictor $\mathcal{I}^{\text{Priv}}$, and the HSP variant that uses this predictor as *HSP-Priv* (HSP-Class in [3]). Given the teacher predictor $\mathcal{I}^{\text{Priv}}$, we can distill $\mathcal{I}_{\theta_i}$ by minimizing the mean squared error to the teacher predicted pose on the motion planning trajectory states $s \in \tau_{\text{MP}}$. We initialize $\mathcal{I}_{\theta_i}$ by sampling trajectories τ_{MP} using $\mathcal{I}^{\text{Priv}}$ as pose predictor. With a reasonably accurate $\mathcal{I}_{\theta_i}$, we then perform online distillation by sampling τ_{MP} directly with $\mathcal{I}_{\theta_i}$ to bridge the distribution gap.

B. Skill Policies

A skill policy π_i controls the system for the duration of the skill until the termination condition is satisfied. We train π_i using episodic RL. The RL reset distribution is the distribution of final states following motion planner execution to reach an initiation pose (Sec. IV-A). The 0-1 reward function $r_i(s) := [s \in \mathcal{T}_i]$ is determined by the ground-truth termination condition $\mathcal{T}_i$ (Sec. IV-C).

Although skill learning can be modeled as a standard RL task, the sparse termination condition reward presents exploration challenges, which can even lead to complete learning failure [9, 27–29]. In light of this, we adopt a residual RL regime [9, 19, 30], starting from a base policy and then fine-tuning it by training an RL agent that outputs differences to the base policy actions, named the

residual policy. Specifically, let π^{base} be the base policy and $\pi_{\theta_i}^{\text{res}}$ be the residual policy to learn. We structure our skill policy as: $\pi_{\theta_i}(o_t) = \pi_i^{\text{base}}(o_t) + \pi_{\theta_i}^{\text{res}}(o_t)$. We then train the residual policy with off-the-shelf reinforcement learning algorithms to maximize the regularized objective: $\mathcal{J}(\theta_i) = \mathbb{E}_{\tau_{\text{skill}} \sim \pi_{\theta_i}} [T_i(s_t) - \alpha \cdot \|\pi_{\theta_i}^{\text{res}}(o_t)\|^2]$. Here, α is a coefficient controlling the regularization strength, which constrains deviations between the base and the fine-tuned policy [9] by penalizing the squared L2 norm of the residual actions; T_i is the termination condition for skill fine-tuning (c.f. Sec. IV-C). We choose the skill policy in HSP (Sec. III-B) as our base policy, which is a behavior-cloning agent trained from the dataset obtained with object-centric data generation.

C. Termination Classification

A skill termination condition $\mathcal{T}_i$ determines whether the goal of the current stage has been achieved (i.e., current state $s \in \mathcal{T}_i$) so that the agent can either enter the next stage or terminate. For deployment, we train a parameterized termination predictor $\mathcal{T}_{\theta_i} : \mathcal{O} \rightarrow \{0, 1\}$ from ReinforceGen rollouts with a binary cross entropy loss using ground-truth labels $[s_t \in \mathcal{T}_i]$ along skill trajectory τ_{skill} .

Compared with the initiation pose predictor in Sec. IV-A, an important distinction is that we do not assume limited observability for the termination predictor during training. We use a hand-crafted termination for skill RL training because RL constantly updates the skill policy, which changes the state distribution for the termination predictor. This leads to out-of-distribution predictions that can be exploited by RL.

Termination fine-tuning. A termination predictor can make two types of errors: false positives, where a termination is predicted but the subtask is not completed, and false negatives, where a subtask is completed but the predictor fails to declare so. False negatives result in a stricter training condition for the RL agent, but empirically have a limited impact on performance because the policy can still explore other states until the predictor produces a true positive. However, false positives can lead to *irrecoverable* states (App. Fig. 4c) and can be exploited by RL training.

Although we could formulate termination prediction itself as an RL problem with a binary action space, we take the following simpler strategy to reduce false positives. Let $T_i : \mathcal{S} \rightarrow \{0, 1\}$ be the termination conditions used for skill fine-tuning. Let θ_i^* be the parameters fine-tuned with T_i . We train a predictor $p_i : \mathcal{S} \rightarrow [0, 1]$ to estimate the probability of task success should we terminate the stage at the current state. Formally, p_i is trained to fit $P(s_T \in \mathcal{T} \mid T_i(s_t) = 1, \tau \sim \pi_{\theta_i})$, $\mathcal{T}$ being the task completion condition. Under the RL formulation of stage terminations, p_i represents the Q-function with T_i as the policy. Let $\mathcal{T}_i^r : \mathcal{S} \rightarrow \{0, 1\}$ be the hand-crafted termination conditions and let T_i initialized to $\mathcal{T}_i^r$. Instead of doing RL exploration for an optimal T_i , we use a greedy solution by rejecting terminations with a Q-function (i.e., p_i) too low. The new termination condition with a rejection threshold ϵ_{term} is: $T_i(s) := \mathcal{T}_i^r(s) \cdot [p_i(s) > \epsilon_{\text{term}}]$.

D. ReinforceGen Hybrid Policy

Algorithm 1 ReinforceGen Deployment Pseudocode

```

1: procedure REINFORCEGEN
2:    $o \leftarrow \text{env.reset}()$  ▷ Get initial observation
3:   for  $i := 1 \rightarrow n$  do
4:      $\langle R_i, \mathcal{I}_{\theta_i}, \pi_{\theta_i}, \mathcal{T}_{\theta_i} \rangle \leftarrow \psi_{\theta_i}$  ▷ Skill  $\psi_i$ 
5:      $p \leftarrow \mathcal{I}_{\theta_i}(o)$  ▷ Predict initiation pose
6:      $\tau \leftarrow \text{planToPose}(\text{env}, p)$  ▷ Motion planning
7:     for  $a \in \tau$  do
8:        $o \leftarrow \text{env.step}(a)$  ▷ Execute motion action
9:        $p' \leftarrow \mathcal{I}_{\theta_i}(o)$ 
10:      if  $\text{dis}(p, p') > \epsilon$  then ▷ Refined initiation prediction
11:         $p \leftarrow p'$ 
12:       $\tau \leftarrow \text{planToPose}(\text{env}, p)$  ▷ Replan trajectory
13:      while  $\mathcal{T}_{\theta_i}(o) \neq 1$  do ▷ Until subgoal success
14:         $a \leftarrow \pi_{\theta_i}(o)$ 
15:         $o \leftarrow \text{env.step}(a)$  ▷ Execute policy action

```

After learning skill initiation conditions, policies, and termination conditions, we can deploy these skills in sequence, using motion planning to connect adjacent pairs of skills. ReinforceGen iterates through each of the n parameterized skills ψ_{θ_i} . For the i -th skill, it predicts an end-effector skill initialization pose p from the most recent observation o (Sec. IV-A). Using the motion planner, it plans a trajectory τ to reach this pose and executes it, while updating the pose target prediction during execution. When the difference between the updated pose and the current motion planner pose exceeds a threshold, ReinforceGen triggers the motion planner to replan (Sec. IV-A). Next, it controls the system using the fine-tuned skill policy π_{θ_i} (Sec. IV-B) by predicting and executing actions a until it reaches an observation o that is predicted to correspond to a termination state (Sec. IV-C). We present the pseudocode for this process in Alg. 1.

E. End-to-End Distillation

The hybrid policy in Sec. IV-D leverages a motion planner to stitch together skill segments. Motion planning requires a model of the world’s collision volume, which can be object meshes if the world is fully observable or a point cloud if it is not [31]. In some deployments, we may wish to bypass the motion planner due to these additional requirements and learn a single end-to-end visuomotor policy. In ReinforceGen, we have the flexibility to do either, where there is a tradeoff between additional requirements and learning difficulty, which a user can tailor to their application.

To train an end-to-end policy, we first configure the motion planner to use the same controller as the skill policy, i.e., by switching from joint-space to task-space control. Then, we generate consistent end-to-end trajectories by stitching together the motion planning and skill policy segments.

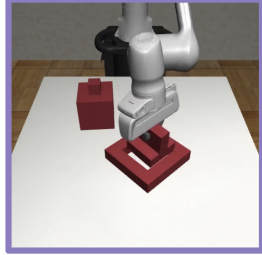
Compared to using baseline imitation agents as the generator, incorporating online data allows ReinforceGen to produce higher-quality source trajectories. Moreover, the random exploration and exploitation process in RL-based fine-tuning makes ReinforceGen agents naturally more resistant to deviations from optimal behaviors. This is especially important in the end-to-end distillation setting, since the deviations compound throughout the long-horizon execution.

Observation	Success Rate (%)	Nut Assem.	Threading	Three Piece	Coffee	Coffee Prep.	Overall
Privileged	SPIRE (Zhou et al.)	N/A	N/A	86.00	98.00	84.00	N/A
Privileged	HSP-Priv (Garrett et al.)	86.20	54.27	70.77	88.12	65.60	72.99
	ReinforceGen-Priv (Ours)	87.20	89.40	82.24	97.03	77.80	86.66
Visual-Proprio.	HSP (Garrett et al.)	40.52	50.20	41.52	55.38	35.80	44.68
	HSP + Replan	78.40	49.80	65.80	80.20	60.20	66.88
	ReinforceGen (Ours)	85.80	82.20	80.40	93.81	80.80	84.60

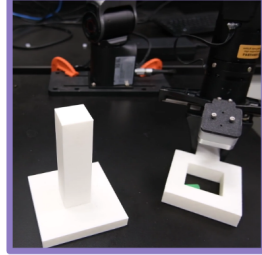
TABLE I: Comparison of task success rates across all tasks. SPIRE uses 200 demonstrations while the rest use 10. SPIRE, HSP-Priv (Sec. IV-A), and ReinforceGen-Priv use privileged state information, while the rest rely only on observations. The numbers are averaged from over 500 rollouts except for SPIRE, which uses 50.



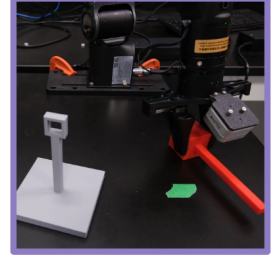
(a) Threading (Stage 2) - **82.2%**



(b) Three Piece (Stage 2) - **93.8%**



(c) Square (Real) - **70%**



(d) Threading (Real) - **35%**

Fig. 6: ReinforceGen agents complete high-precision skills with high success rates (numbers shown in bold).

V. EXPERIMENTS

Tasks. We benchmark ReinforceGen on several robot-suite [32] manipulation tasks, specifically the D2 variants [3] (largest initialization range) of the *Nut Assembly*, *Threading*, *Three Piece*, *Coffee*, and *Coffee Preparation* tasks. Additionally, we experiment on two real-robot tasks to investigate ReinforceGen’s capacity for real-world deployment. See Appendix II for more detail.

Observation space. The observation space comprises 84×84 RGB images from a front-view camera and a robot wrist camera as well as robot proprioception, namely its end-effector pose and gripper joint position. For the *Threading* task, we include an additional side-view camera.

Demonstrations. We collected 10 human source demonstrations per task as source and adapt them into a BC dataset of 1,000 demonstrations with ReinforceGen.

Baselines. We use HSP-Priv (Sec. IV-A, [3]) as our baseline with privileged information. We further fine-tune its skill policies to establish an upper-bound on performance (ReinforceGen-Priv). For baselines sharing the same assumptions, we use HSP. We further compare ReinforceGen with two LLM-based no-demonstration baselines: RoboGen [33] and CaP-X [34]. For CaP-X, we use single-turn CaP-Agent0 with non-privileged vision APIs.

Details. We skip fine-tuning on stages that have high baseline success rates to save computational cost. For an exhaustive implementation list, refer to App. III-D. We use hand-crafted stage terminations in our evaluations for all hybrid policies despite it requiring state information, which is in line with prior works [3, 9]. We instead include an ablation that uses learned terminations.

A. Simulated Results

ReinforceGen is proficient in all tasks despite partial observability. Tab. I shows that ReinforceGen achieves over

80% task completion rate in all our tested tasks, including long-horizon tasks with as many as 5 stages, all while relying only on camera sensory input and proprioception information.

ReinforceGen achieves an 89% relative performance increase over baselines without state observability on average. In Tab. I, comparing HSP-Priv and HSP, we observe a significant drop in success rates due to the lack of object location awareness. With the same accessible information, ReinforceGen nearly doubles HSP’s overall success rate. We also notice that our gains are especially large in longer-horizon tasks with 4-5 stages, i.e., *Nut Assembly*, *Three Piece*, *Coffee Preparation*, reaching **109%** relative increase.

ReinforceGen is competitive with baselines with privileged state information. We fine-tuned HSP-Priv with access to state information as a pseudo-performance upper bound (ReinforceGen-Priv in Tab. I). Despite the disadvantages, ReinforceGen only shows a maximum deficit of **8%** in *Threading*, with **2%** decrease in the overall success rate.

ReinforceGen solves complex tasks with nontrivial interactions. We evaluate ReinforceGen on a set of tasks with high-precision skills and high object initialization variances. To establish this, we benchmark two baselines with LLM-based skill learning in Tab. II. Both baselines struggle to capture the precise interactions required to solve the tasks. Notably, RoboGen agent reaches 100% success rate in *Nut Assembly* by exploiting the imperfect stage termination, which results in the failure of future stages. In contrast, ReinforceGen learns nearly perfect skills seeded from a small amount of demonstrations.

Pose target replanning improves motion planning reaching accuracy and skill completion. When comparing HSP and HSP+Replan in Table I, we find that adding replanning increases the success rate of HSP by **22%** (from 44.68% to 66.88%), a relative **50%** improvement. We present a detailed case study in *Nut Assembly* in Appendix IV-A.

Success Rate (%)	Nut Assembly 1st / Full	Threading 1st / Full	Three Piece 1st / Full	Coffee 1st / Full
RoboGen	100 / 0	4 / 0	41 / 0	26 / 0
CaP-X	7 / 0	7 / 0	7 / 0	36 / 0
ReinforceGen	97 / 86	100 / 82	100 / 80	100 / 94

TABLE II: Comparing the first stage and full task success rates of RoboGen and CaP-X with ReinforceGen. Both RoboGen and CaP-X struggle to solve the even first stages of most tasks.

Skill policy fine-tuning recovers inaccurate starting poses and suboptimal imitated skills. We ablate skill fine-tuning from ReinforceGen to show its effectiveness in Tab. III. On average, skill fine-tuning improves task completion rates by **24.41%** while using **9.74%** fewer steps to complete. Skill fine-tuning is most effective in the second stage of *Threading*, which is the most precision-demanding stage among all benchmarked tasks, and where the IL agent has the lowest success rate.

Improvement (%)	Nut Assembly	Threading	Three Piece	Coffee
Success Rate	4.62	65.73	10.55	16.74
Efficiency	8.43	16.39	10.18	3.97

TABLE III: ReinforceGen’s policy improvement through skill fine-tuning. Efficiency is the average number of timesteps to complete each skill. Success Rate and Efficiency are averaged over all stages.

Termination fine-tuning repairs cross-stage causal effects. When directly applied to HSP-Priv, termination fine-tuning improves its success rate from 66.44% to 70.77%; after fine-tuning skills with the new terminations, the success rate further increases from 72.60% to 82.24%.

ReinforceGen can train end-to-end policies. We also distill the hybrid ReinforceGen agents to end-to-end visuomotor policies. We first configure the motion planner’s action space to be consistent with the skill policy, allowing us to stitch the connect and skill phase trajectories of each stage together. We then create a dataset of such rollouts and train a BC agent from them. The resulting success rates are shown in Table IV. In *Threading* and *Coffee*, ReinforceGen distills to proficient end-to-end agents, with significant improvement over the baseline. However, in *Nut Assembly* and *Three Piece*, both agents struggle to complete the task. We hypothesize that these tasks require long transit motions, which exacerbate the difficulty of learning when compared to learning just contact-rich motion.

Success Rate (%)	Nut Assembly	Threading	Three Piece	Coffee
HSP-Priv	35.00	60.40	20.20	84.60
ReinforceGen-Priv	28.00	83.60	18.60	94.20

TABLE IV: End-to-end distillation results using datasets generated with ReinforceGen-Priv and HSP-Priv.

B. Real World Evaluation

We demonstrate the real-world deployment capability of ReinforceGen on a variant of the *Square* task (Fig. 1a, 6c) using a 6-DOF AgileX PiPER arm and a RealSense D435i camera. We collect 10 source human teleoperation demonstrations with a real robot and replay the trajectories in

simulation, using FoundationPose [35] to estimate the initial object state. Because we already assume pose estimation for real-to-sim demonstration transfer, we skipped initialization pose prediction (Sec. IV-A) and instead simply adapted ground-truth demonstration initialization poses. Agents are evaluated for 20 rollouts. An additional evaluation on the real-world *Threading* task (Fig. 1b, 6d) is presented in App. IV-D. See our website for video demonstrations.

ReinforceGen successfully transfers to real robots. From Table V, ReinforceGen agents reach a **70%** success rate in the real world, a **2.8×** relative performance improvement compared to our BC baseline (HSP), which is consistent with the results of the more expansive simulation experiments.

Termination fine-tuning reduces reward exploitation in skill fine-tuning. We ablate ReinforceGen with respect to termination fine-tuning (Sec. IV-C). In simulation, ReinforceGen completes grasping stages with almost perfect success rates at the cost of drastic decreases in second stage success rates. Tab. V shows this also occurs in the real world, as we see lower sim-to-real transfer compared to HSP, and eventually a regression on real-world success rate. We further evaluate a version with hand-crafted conditions for stable grasps (Term. w/ Human FT) that achieves almost perfect sim-to-real transfer, which demonstrates the importance of termination fine-tuning.

Success Rate (%)	Sim	Real	Sim-to-Real
HSP	40.00	25.00	62.50
ReinforceGen (w/o Term. FT)	45.00	15.00	33.33
ReinforceGen	90.00	70.00	77.78
ReinforceGen (w/ Human Term. FT)	90.00	85.00	94.44

TABLE V: Real-world evaluation results. Success rates are averaged over 20 rollouts. Sim-to-real transfer success rates are calculated by dividing real-world success rate by simulation success rate.

VI. CONCLUSION

We presented ReinforceGen, an automated demonstration generation system that integrates planning, demonstration adaptation, and reinforcement learning to bootstrap a small set of human demonstrations into a large high-quality dataset. These demonstrations can be used to train end-to-end imitation learners or atomic skill policies that are stitched together using motion planning, demonstrating the flexibility of our approach. The use of reinforcement learning for exploration improves policy success rates when compared to prior works due to its ability to explore beyond adapted demonstrations.

We disclose the use of Claude and ChatGPT to assist with code implementation and proofreading the manuscript.

REFERENCES

- [1] A. Mandlekar, D. Xu, J. Wong, S. Nasiriany, C. Wang, R. Kulkarni, L. Fei-Fei, S. Savarese, Y. Zhu, and R. Martín-Martín, “What matters in learning from offline human demonstrations for robot manipulation,” in *Conference on Robot Learning (CoRL)*, 2021.
- [2] A. Mandlekar, S. Nasiriany, B. Wen, I. Akinola, Y. Narang, L. Fan, Y. Zhu, and D. Fox, “Mimicgen: A data generation system for scalable robot learning using human demonstrations,” in *Conference on Robot Learning (CoRL)*, 2023.

- [3] C. R. Garrett, A. Mandlekar, B. Wen, and D. Fox, "Skillmimicgen: Automated demonstration generation for efficient skill learning and deployment," in *8th Annual Conference on Robot Learning*, 2024. [Online]. Available: <https://openreview.net/forum?id=YOFrRTDC6d>
- [4] Z. Jiang, Y. Xie, K. Lin, Z. Xu, W. Wan, A. Mandlekar, L. Fan, and Y. Zhu, "Dexmimicgen: Automated data generation for bimanual dexterous manipulation via imitation learning," in *2025 IEEE International Conference on Robotics and Automation (ICRA)*, 2025.
- [5] M. J. McDonald and D. Hadfield-Menell, "Guided imitation of task and motion planning," in *Conference on Robot Learning*. PMLR, 2022, pp. 630–640.
- [6] M. Dalal, A. Mandlekar, C. Garrett, A. Handa, R. Salakhutdinov, and D. Fox, "Imitating task and motion planning with visuomotor transformers," *arXiv preprint arXiv:2305.16309*, 2023.
- [7] C. R. Garrett, R. Chitnis, R. Holladay, B. Kim, T. Silver, L. P. Kaelbling, and T. Lozano-Pérez, "Integrated task and motion planning," *Annual review of control, robotics, and autonomous systems*, vol. 4, pp. 265–293, 2021.
- [8] A. Mandlekar, C. Garrett, D. Xu, and D. Fox, "Human-in-the-loop task and motion planning for imitation learning," in *7th Annual Conference on Robot Learning*, 2023.
- [9] Z. Zhou, A. Garg, D. Fox, C. R. Garrett, and A. Mandlekar, "SPIRE: Synergistic planning, imitation, and reinforcement learning for long-horizon manipulation," in *8th Annual Conference on Robot Learning*, 2024. [Online]. Available: <https://openreview.net/forum?id=cvUXoou8iz>
- [10] P. Florence, C. Lynch, A. Zeng, O. A. Ramirez, A. Wahid, L. Downs, A. Wong, J. Lee, I. Mordatch, and J. Tompson, "Implicit behavioral cloning," in *Conference on Robot Learning*. PMLR, 2022, pp. 158–168.
- [11] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. Burchfiel, and S. Song, "Diffusion policy: Visuomotor policy learning via action diffusion," *arXiv preprint arXiv:2303.04137*, 2023.
- [12] N. M. Shafiuallah, Z. Cui, A. A. Altanzaya, and L. Pinto, "Behavior transformers: Cloning k modes with one stone," *Advances in neural information processing systems*, vol. 35, pp. 22955–22968, 2022.
- [13] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, "Learning fine-grained bimanual manipulation with low-cost hardware," 2023.
- [14] M. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, *et al.*, "Openvla: An open-source vision-language-action model," *arXiv preprint arXiv:2406.09246*, 2024.
- [15] C. Tang, B. Abbatematteo, J. Hu, R. Chandra, R. Martín-Martín, and P. Stone, "Deep reinforcement learning for robotics: A survey of real-world successes," *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 8, no. Volume 8, 2025, pp. 153–188, 2025. [Online]. Available: <https://www.annualreviews.org/content/journals/10.1146/annurev-control-030323-022510>
- [16] P. Wu, A. Escontrela, D. Hafner, K. Goldberg, and P. Abbeel, "Daydreamer: World models for physical robot learning," in *Conference on Robot Learning*, 2022. [Online]. Available: <https://api.semanticscholar.org/CorpusID:250088882>
- [17] W. Yu, N. Gileadi, C. Fu, S. Kirmani, K.-H. Lee, M. Gonzalez Arenas, H.-T. Lewis Chiang, T. Erez, L. Hasenclever, J. Humpalik, *et al.*, "Language to rewards for robotic skill synthesis," *Arxiv preprint arXiv:2306.08647*, 2023.
- [18] A. Nair, B. McGrew, M. Andrychowicz, W. Zaremba, and P. Abbeel, "Overcoming exploration in reinforcement learning with demonstrations," *2018 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 6292–6299, 2017.
- [19] T. Johannink, S. Bahl, A. Nair, J. Luo, A. Kumar, M. Loskyll, J. A. Ojea, E. Solowjow, and S. Levine, "Residual reinforcement learning for robot control," *2019 International Conference on Robotics and Automation (ICRA)*, pp. 6023–6029, 2018.
- [20] M. Dalal, T. Chiruvolu, D. Chaplot, and R. Salakhutdinov, "Plan-seq-learn: Language model guided rl for solving long horizon robotics tasks," 2024.
- [21] Z. Xue, S. Deng, Z. Chen, Y. Wang, Z. Yuan, and H. Xu, "Demogen: Synthetic demonstration generation for data-efficient visuomotor policy learning," *arXiv preprint arXiv:2502.16932*, 2025.
- [22] K. Lin, V. Rangunath, A. McAlinden, A. Prasad, J. Wu, Y. Zhu, and J. Bohg, "Constraint-preserving data generation for visuomotor policy learning," *arXiv preprint arXiv:2508.03944*, 2025.
- [23] C. Li, M. Xu, A. Bahety, H. Yin, Y. Jiang, H. Huang, J. Wong, S. Garlanka, C. Gokmen, R. Zhang, *et al.*, "Momagen: Generating demonstrations under soft and hard constraints for multi-step bimanual mobile manipulation," in *RSS 2025 Workshop on Whole-body Control and Bimanual Manipulation: Applications in Humanoids and Beyond*.
- [24] Z. Wang, C. R. Garrett, L. P. Kaelbling, and T. Lozano-Pérez, "Learning compositional models of robot skills for task and motion planning," *The International Journal of Robotics Research*, vol. 40, no. 6–7, pp. 866–894, 2021.
- [25] S. Cheng, C. R. Garrett, A. Mandlekar, and D. Xu, "Nod-tamp: Generalizable long-horizon planning with neural object descriptors." Proceedings of The 8th Conference on Robot Learning, 2024.
- [26] M. Stolle and D. Precup, "Learning options in reinforcement learning," in *Symposium on Abstraction, Reformulation and Approximation*, 2002. [Online]. Available: <https://api.semanticscholar.org/CorpusID:16398811>
- [27] B. C. Stadie, S. Levine, and P. Abbeel, "Incentivizing exploration in reinforcement learning with deep predictive models," *ArXiv*, vol. abs/1507.00814, 2015.
- [28] H. Tang, R. Houthoofd, D. Foote, A. Stooke, X. Chen, Y. Duan, J. Schulman, F. D. Turck, and P. Abbeel, "#exploration: A study of count-based exploration for deep reinforcement learning," *NIPS*, 2017.
- [29] A. Ecoffet, J. Huizinga, J. Lehman, K. O. Stanley, and J. Clune, "First return then explore," *Nature*, vol. 590 7847, pp. 580–586, 2021.
- [30] T. Silver, K. R. Allen, J. B. Tenenbaum, and L. P. Kaelbling, "Residual policy learning," *ArXiv*, vol. abs/1812.06298, 2018.
- [31] B. Sundaralingam, S. K. S. Hari, A. Fishman, C. Garrett, K. Van Wyk, V. Blukis, A. Millane, H. Oleynikova, A. Handa, F. Ramos, *et al.*, "Curobo: Parallelized collision-free robot motion generation," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2023. [Online]. Available: <https://arxiv.org/pdf/2310.17274>
- [32] Y. Zhu, J. Wong, A. Mandlekar, and R. Martín-Martín, "robosuite: A modular simulation framework and benchmark for robot learning," in *arXiv preprint arXiv:2009.12293*, 2020.
- [33] Y. Wang, Z. Xian, F. Chen, T.-H. Wang, Y. Wang, K. Fragkiadaki, Z. Erickson, D. Held, and C. Gan, "Robogen: Towards unleashing infinite data for automated robot learning via generative simulation," *arXiv preprint arXiv:2311.01455*, 2023.
- [34] M. Fu, J. Yu, K. El-Refai, E. Kou, H. Xue, H. Huang, W. Xiao, G. Wang, F.-F. Li, G. Shi, *et al.*, "CaP-X: A framework for benchmarking and improving coding agents for robot manipulation," *arXiv preprint arXiv:2603.22435*, 2025. [Online]. Available: <https://arxiv.org/abs/2603.22435>
- [35] W. Bowen, W. Yang, J. Kautz, and S. Birchfield, "FoundationPose: Unified 6d pose estimation and tracking of novel objects," in *CVPR*, 2024.
- [36] D. Yarats, R. Fergus, A. Lazaric, and L. Pinto, "Mastering visual continuous control: Improved data-augmented reinforcement learning," *arXiv preprint arXiv:2107.09645*, 2021.

APPENDIX I

OVERVIEW

- Appendix II shows the details of the benchmark task set.
- Appendix III lists the details to reproduce our results.
- Appendix IV includes additional experiment results.
- Appendix V includes additional implementation details.
- Appendix VI provides details on our real-world evaluation setup.

APPENDIX II

TASKS

A. Simulation Tasks

We choose the five tasks from Robosuite [3, 32] (Fig. 7). We use the largest initiation range version (D2) for all tasks. The only exception is *Nut Assembly*, where we reduce the x -range of the nuts' placement to $(-0.15, 0.15)$ since the original range produces unreachable initial positions.

Observations. The observation consists of two (or three, in Threading) 84×84 RGB images and robot proprioception state, including a 6-dim end-effector pose composed of 3-dim Euclidean position in meters and 3-dim rotation represented in axis-angle form; a 2-dim gripper position, representing the distances of the two grippers to the center; and a 7-dimensional joint configuration. The front-view camera is demonstrated in Figure 7; the wrist camera angle is shown in Figure 6b (right); the extra Threading side-view camera is shown in Figure 6d (right).

B. Real-World Tasks

Square (first two stages of *Nut Assembly*) and *Threading* are selected for real-world evaluations (Fig. 8). We remove the legs of the ring stand in *Threading* for 3D-printing compatibility. We also use a limited initialization range due to the size and controllability of the robot arm. The specific ranges are:

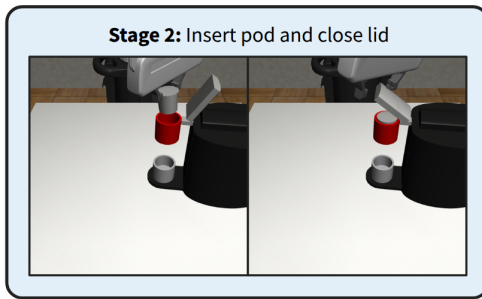
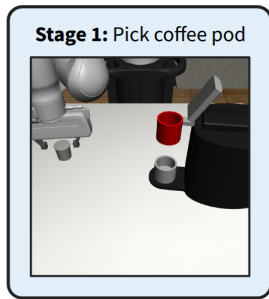
Square

- Square nut: $x \in (-0.05, 0)$, $y \in (0.05, 0.1)$, $z_{\text{rot}} \in (-30^\circ, 30^\circ)$
- Square peg: $x \in (-0.05, 0)$, $y \in (-0.1, -0.05)$, $z_{\text{rot}} \in (-30^\circ, 30^\circ)$

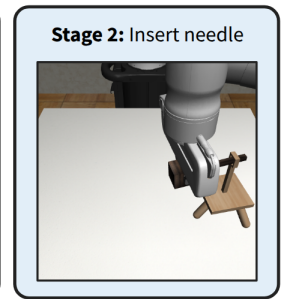
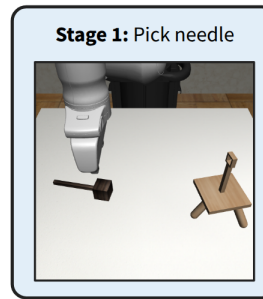
Threading

- Needle: $x \in (-0.05, 0)$, $y \in (0.05, 0.1)$, $z_{\text{rot}} \in (-30^\circ, 30^\circ)$
- Ring stand: $x \in (-0.025, 0.025)$, $y \in (-0.1, -0.05)$, $z_{\text{rot}} \in (-15^\circ, 15^\circ)$

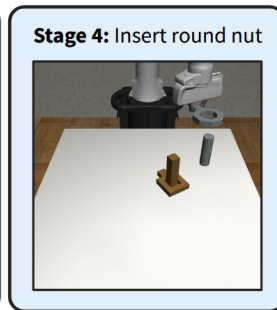
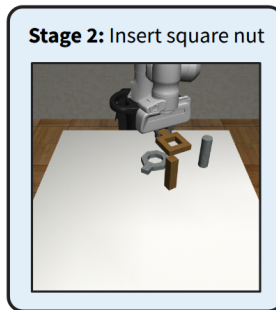
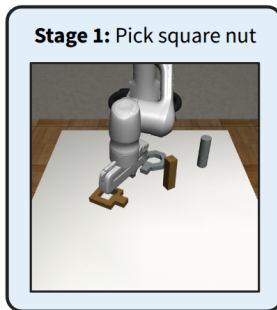
Observations. For simulation training, we use the same observation space as our simulation task setup. We do not include the third side-viewing camera in *Threading*, as the original two cameras suffice in this initialization setting. The robot joint configuration input also becomes 6-dimensional to accommodate the 6-DOF arm used in our evaluation.



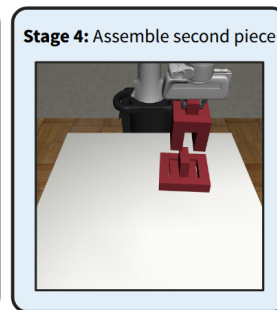
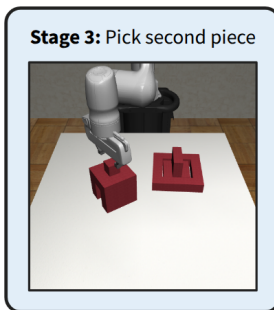
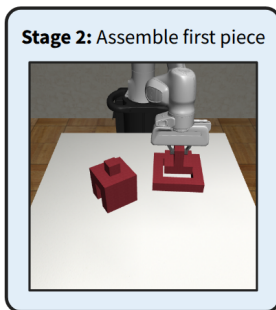
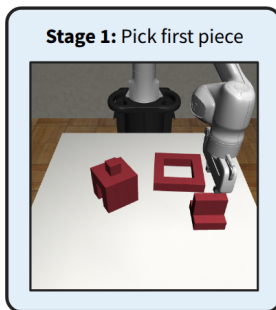
(a) Coffee (2 stages)



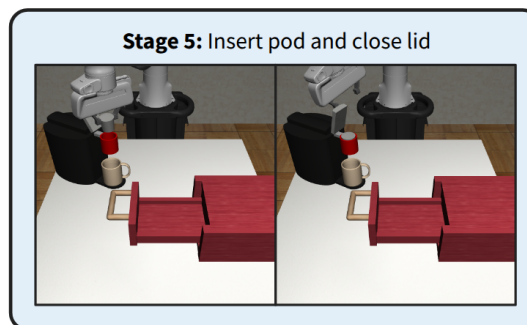
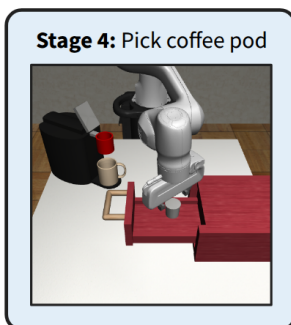
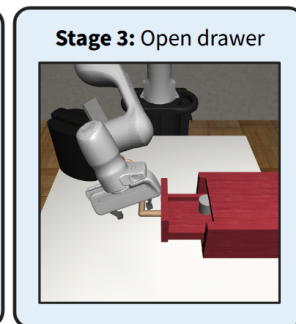
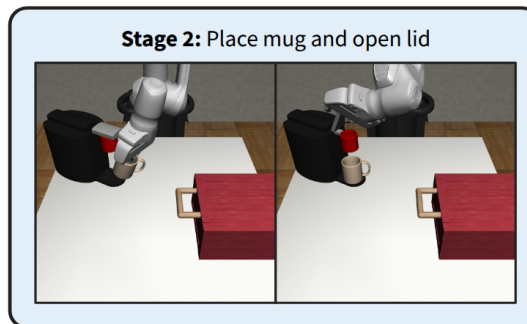
(b) Threading (2 stages)



(c) Nut Assembly (4 stages)

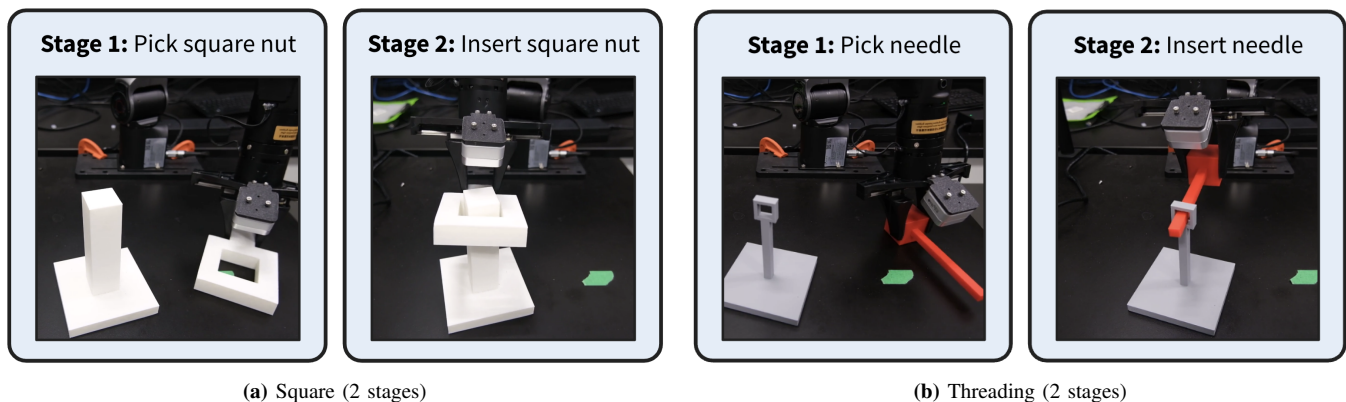


(d) Three Piece Assembly (4 stages)



(e) Coffee Preparation (5 stages)

Fig. 7: All five simulation tasks showcased.



(a) Square (2 stages)

(b) Threading (2 stages)

Fig. 8: Two real-world tasks.

APPENDIX III
REPRODUCIBILITY

A. Thresholds Used in ReinforceGen

Threshold for motion planner replanning (Sec. IV-A)

We replan the motion planning trajectory when the pose distance (c.f. App. V-B) between the newest predicted pose and the current pose target exceeds **0.05**.

Threshold for termination rejection (Sec. IV-C)

We reject terminations with a predicted task completion rate lower than **0.4**.

B. Hyperparameters for Data Generation and Imitation Learning

We follow the exact setup in [3].

C. Hyperparameters for Reinforcement Learning

TABLE VI: DrQ-v2 hyperparameters.

Network structure	CNN
Learning rate	1e-4
Discount	0.99
Batch size	256
n -step returns	3
Action repeat	1
Seed frames	4000
Feature dim	50
Hidden dim	1024
Optimizer	Adam
Training steps	2M
Constraint coef. (α)	5.0

We use DrQ-v2 [36] for skill fine-tuning (Sec. IV-B). The hyperparameters are shown in Tab. VI.

D. Usage of Fine-tuning Methods

By default, all stages in all tasks use first-iteration pose distillation and real-time replanning (Sec. IV-A). The usage of second-iteration pose distillation and skill/termination fine-tuning is listed in Tab. VII.

E. End-to-End Distillation

We add Gaussian noise $\mathcal{N}(0, \sigma^2)$ with $\sigma = 0.01$ during trajectory generation. For each task, we generate 3000 successful trajectories. We use the same hyperparameters as skill imitation for IL.

TABLE VII: Usage of different fine-tuning methods in ReinforceGen

Task	Stage	Pose distillation (Sec. IV-A)	Skill fine-tune (Sec. IV-B)	Termination fine-tune (Sec. IV-C)
Coffee	1	$\times$	$\times$	$\times$
	2	$\times$	$\checkmark$	$\times$
Threading	1	$\times$	$\times$	$\times$
	2	$\times$	$\checkmark$	$\times$
Nut Assembly	1	$\checkmark$	$\times$	$\times$
	2	$\times$	$\checkmark$	$\times$
	3	$\checkmark$	$\checkmark$	$\times$
	4	$\times$	$\times$	$\times$
Three Piece	1	$\times$	$\times$	$\checkmark$
	2	$\times$	$\checkmark$	$\checkmark$
	3	$\times$	$\times$	$\checkmark$
	4	$\times$	$\checkmark$	$\checkmark$
Coffee Prep.	1	$\times$	$\times$	$\times$
	2	$\times$	$\checkmark$	$\times$
	3	$\times$	$\checkmark$	$\times$
	4	$\times$	$\checkmark$	$\times$
	5	$\times$	$\checkmark$	$\times$

APPENDIX IV ADDITIONAL EXPERIMENTS

A. Case Study on the Effectiveness of Replanning in Nut Assembly

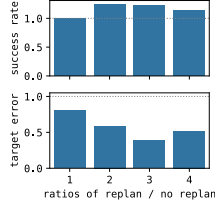


Fig. 9: Ablation of real-time replanning at every stage of *Nut Assembly*. The top figure shows the per-stage success rate, and the bottom shows the pose target error. The numbers are ratios of applying replanning versus not applying.

To illustrate the effectiveness of real-time replanning (Sec. IV-A), we plot the reduction in prediction error and improvement in task success rate with replanning in all 4 stages in *Nut Assembly*. The results in Fig. 9 show that replanning significantly reduces the target prediction error in all stages, and in turn, improves the subsequent skill success rates.

B. Learned Termination Predictor

Learned termination predictors have limited impact on task completion. We evaluate ReinforceGen with a learned termination predictor that determines termination with partial observations. As shown in Table VIII, using a learned termination predictor only introduces minor drops in success rates in *Threading* and *Three Piece*, while maintaining its performance in the other tasks.

C. Ablation on Distillation

In Fig. 10, we add artificial noise to the actions and plot the relative performance decreases against the noise scale. In all tasks, ReinforceGen appears to be more resistant to action noise and trajectory deviations, matching our assertions in Sec. IV-E.

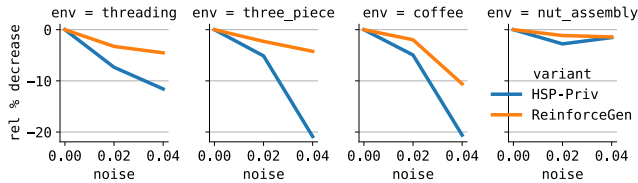


Fig. 10: Comparing the resistance to action noise between HSP-Priv and ReinforceGen. For most tasks, ReinforceGen has a much higher tolerance to action noise.

D. Real-World Evaluation on Threading

We also evaluated ReinforceGen on a real-world version of *Threading* (see Sec. II-B). We collected only 1 tele-op demonstration and generated 100 demonstrations for imitation learning to test the limit of ReinforceGen. We also only

fine-tuned the second stage skill policy since it is the most challenging part of the task. The results are shown in Tab. IX.

Consistent with the conclusion drawn in the *Square* experiments, The ReinforceGen agent not only shows a significantly higher success rate than HSP on a real robot, but also is more effective in handling the sim-to-real gap.

Success Rate (%)	<i>Nut Assembly</i>	<i>Threading</i>	<i>Three Piece</i>	<i>Coffee</i>
Oracle Termination	85.80	82.20	80.40	93.81
Learned Termination	84.60	79.20	73.80	92.60

TABLE VIII: ReinforceGen’s learned termination predictor, which unlike the Oracle Termination does not have access to the state, performs well albeit slightly worse when compared to the Oracle Termination upper bound.

Success Rate (%)	<i>Simulation Success.</i>	<i>Real Success.</i>	<i>Sim-to-Real Success.</i>
HSP	20.00	5.00	25.00
ReinforceGen	75.00	35.00	46.67

TABLE IX: Real-world evaluation results on *Threading*. Success rates are averaged over 20 rollouts. Sim-to-real transfer success rates are calculated by dividing real-world success rate by simulation success rate.

APPENDIX V

ADDITIONAL DETAILS

A. Pose Noise in Section IV-A

In Figure 5, we add artificial noise to initiation poses to demonstrate their relationship with skill completion rates. The noise is added with the following procedure. Let the original pose be $\mathbf{P} := \mathbf{p} \oplus \mathbf{r}$, where $\mathbf{p}$ is a 3-dimensional position vector in meters, $\mathbf{r}$ is a 3-dimensional rotation vector in the axis-angle form. A noisy version of $\mathbf{P}$ with a scale σ is defined as:

$$\tilde{\mathbf{P}} \leftarrow \mathbf{P} + \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I}_6). \quad (1)$$

B. Pose Distance Metric

We use the following metric to compute the distance between two poses throughout our implementation. Let $\mathbf{P}_1 := (\mathbf{p}_1, \mathbf{q}_1)$, $\mathbf{P}_2 := (\mathbf{p}_2, \mathbf{q}_2)$ be the two poses to compare, $\mathbf{p}_1, \mathbf{p}_2$ the Euclidean positions, and $\mathbf{q}_1, \mathbf{q}_2$ the quaternion-form rotations.

$$d^{\text{pos}}(\mathbf{P}_1, \mathbf{P}_2) := \sqrt{(\mathbf{p}_1 - \mathbf{p}_2)^\top (\mathbf{p}_1 - \mathbf{p}_2)} \quad (2)$$

$$d^{\text{rot}}(\mathbf{P}_1, \mathbf{P}_2) := \frac{1}{\pi} \min\{\cos^{-1}(\mathbf{q}_1^\top \mathbf{q}_2), \cos^{-1}(-\mathbf{q}_1^\top \mathbf{q}_2)\} \quad (3)$$

$$d^{\text{pose}}(\mathbf{P}_1, \mathbf{P}_2) := d^{\text{pos}}(\mathbf{P}_1, \mathbf{P}_2) + d^{\text{rot}}(\mathbf{P}_1, \mathbf{P}_2) \quad (4)$$

APPENDIX VI

REAL-WORLD SETUP

We provide additional details on our real-world evaluations. See Sec. II-B for the details of the real-world tasks. See Sec. IV-D for evaluation results on *Threading*.

Robot. We use an AgileX PiPER 6-DOF arm for all our real-world evaluations. The arm is mounted at a 90-degree angle, facing the negative y -axis.

Pose estimation. We use a fixed RealSense D435i camera to capture the scene and FoundationPose [35] to estimate the object poses. We do pose estimation once at the beginning of an episode.

Demonstrations. We use another PiPER teacher arm to tele-op the main arm and collect demonstrations in the real world. We then replay the joint position trajectory of the tele-op demonstrations in a MuJoCo simulator and convert them into a simulation-based demonstration dataset. For *Square*, we collected 10 tele-op demonstrations and generated a 1,000-demonstration dataset. For *Threading*, we collected 1 tele-op demonstration and generated a 100-demonstration dataset. We use the same setup as our simulation evaluations to train a BC policy.

Fine-tuning. Since pose estimation is required in our workflow, we do not need to do object-state-agnostic initiation pose prediction. We therefore use the privileged pose predictor as in HSP-Priv and omit the initiation pose prediction fine-tuning process. We use the same setup as in our simulation experiments for skill and termination fine-tuning. For *Threading*, we omit termination fine-tuning and only fine-tune the second-stage skill policy.

Deployment. We use the following procedure to deploy our simulation-based policy on a real robot. First, we use FoundationPose to estimate the poses of the objects in the real world. We then sync the estimated poses in our MuJoCo simulation and run the evaluated policy in simulation, while recording the joint position and gripper action trajectory. If the simulation is successful, we finally replay the trajectory in the real world by sending joint and gripper commands to the robot arm. See Fig. 1 for an illustration.